

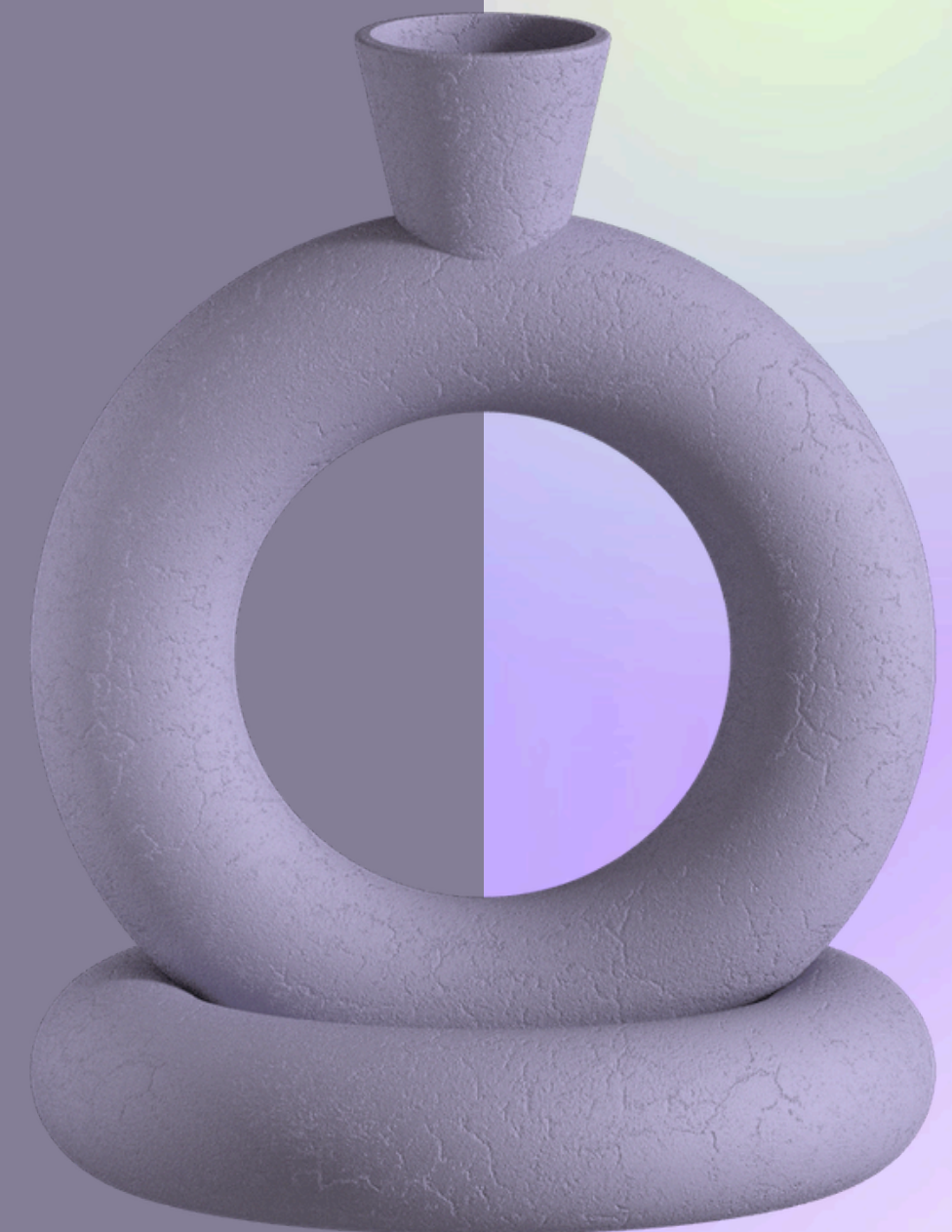
SEMINAR · HIMTI UMT

Vibe Coding

From Business Problem to Production App

Cara membangun aplikasi bersama AI dengan workflow yang benar, dari masalah bisnis nyata hingga deployment ke production.

VIBECODING · FROM BUSINESS PROBLEM TO PRODUCTION APP





Muhamad Rifki Firdaus

Software Engineer • Frontend Specialist • AI & Product Enthusiast

Mahasiswa Teknik Informatika

Universitas Muhammadiyah Tangerang

2024

2027

Software Engineer

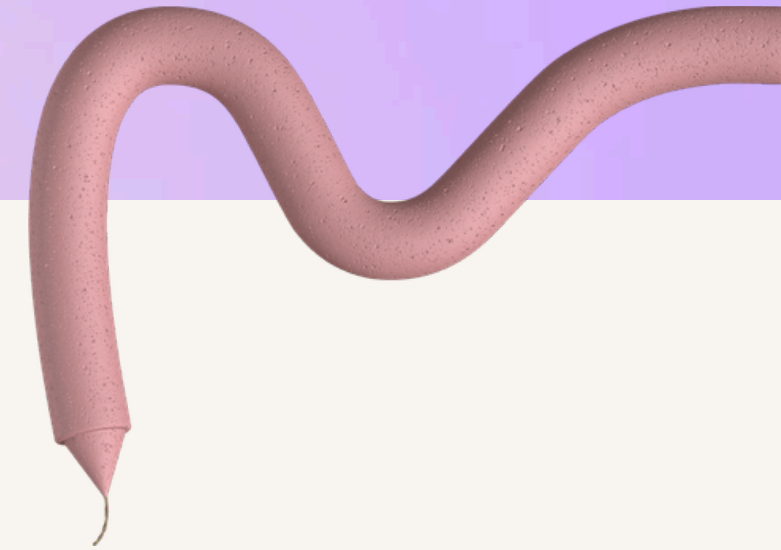
Pengembang Teknologi & Inovasi

Frontend

AI Integration

Roadmap Seminar

Tiga jam, delapan babak. Setengah teori, setengah praktik langsung di laptop Anda.



01	Opening	Menyamakan persepsi soal AI dan coding.
02	What Is Vibecoding	Definisi, evolusi, dan landscape AI hari ini.
03	Master Workflow	Delapan langkah dari masalah ke aplikasi.
04	Business Discovery	Cari masalah bisnis nyata lewat Google Maps.
05	Documentation	Menulis PRD, Scope, Design, dan Task di IDE.
06	Agentic Development	Planner dan Executor membangun aplikasi.
07	Deploy & Audit	Naikkan ke production, lalu bedah hasilnya.
08	Q&A	Diskusi terbuka dan arah belajar berikutnya.

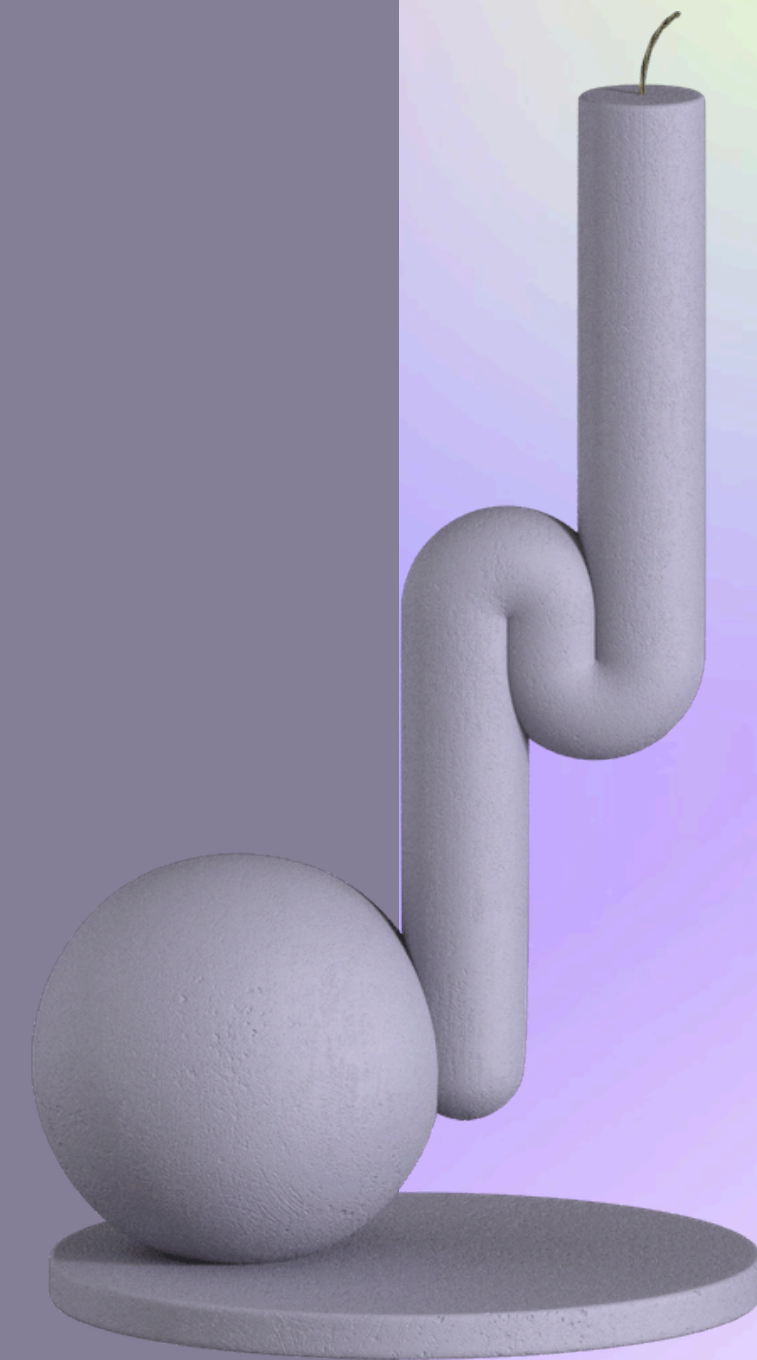
Satu kata pertama saat denger 'Vibe Coding'?



02

What Is Vibe Coding

Menyamakan persepsi: apa yang sebenarnya berubah ketika AI masuk ke proses engineering.



Cara lama membangun software

3-6

BULAN SEBELUM ADA YANG BISA DIPAKAI

Ide muncul, developer langsung menulis kode, lalu berharap hasilnya sesuai kebutuhan. Dokumentasi menyusul belakangan — kalau sempat.

Idea → Coding → Deploy

DI MANA BIAYANYA

Kebutuhan berubah di tengah jalan

Tidak ada dokumen yang disepakati, jadi setiap orang punya versi kebenarannya sendiri.

Scope melebar tanpa disadari

Fitur bertambah satu per satu sampai timeline dan tim ikut berantakan.

Feedback datang terlalu terlambat

Bisnis baru melihat produknya saat hampir selesai, dan sering tidak sesuai.

Evolusi Software Development

AI tidak datang tiba-tiba. Ia bergerak dari sekadar melengkapi kode, sampai mengeksekusi rencana sendiri.

01

Traditional



Manusia menulis 100% kode.

Manusia: penulis kode

02

AI Assisted



AI melengkapi baris di editor.

Manusia: pengarah baris

03

Agentic



AI membaca dokumen, lalu mengeksekusi.

Manusia: reviewer

04

Autonomous



AI mengambil issue dan mengirim PR.

Manusia: pemberi tujuan



Semakin ke kanan, menulis kode makin bukan pekerjaan utama kita. Yang jadi pekerjaan utama adalah spesifikasi dan review.

Landscape AI untuk Developer

Empat kategori dengan cara pakai yang berbeda. Salah memilih kategori, salah juga ekspektasinya.

01

AI Builder

Prompt jadi aplikasi utuh dalam hitungan menit.



v0



Lovable

B

Bolt



Firebase

PROTOTYPE CEPAT

02

AI Assistant

Menemani Anda mengetik kode di dalam editor.



Copilot



Cursor



VS Code

PERCEPAT CODING

03

AI Agent

Membaca dokumen, merencanakan, lalu mengerjakan.



Claude



Gemini



OpenCode



Antigravity

BANGUN FITUR UTUH

04

Autonomous

Menerima issue dan menyelesaikannya sendiri.



Devin



Codex



Open Claw

KERJA TANPA DIAWASI

Salah paham terbesar soal Vibecoding

Vibecoding sering dikira jalan pintas. Padahal yang dipangkas AI adalah waktu mengetik, bukan waktu berpikir.

MITOS

~~Idea~~ → ~~Prompt~~ → ~~Aplikasi Jadi~~ ✗

YANG BENAR-BENAR TERJADI

Scope melebar tanpa rem

AI tidak tahu apa yang TIDAK boleh dibangun, jadi ia membangun semuanya.

Fitur meleset dari kebutuhan

Yang jadi bukan yang dibutuhkan bisnis, tapi yang paling mudah ditebak AI.

Kode sulit dirawat

Struktur berubah tiap sesi karena tidak ada acuan arsitektur yang tetap.

Bug baru ketahuan di production

Tidak ada definisi selesai, jadi tidak ada yang bisa diuji.

Master Workflow

Delapan langkah dari masalah bisnis sampai aplikasi hidup. Setiap langkah menghasilkan output yang menjadi input langkah berikutnya.

FASE 1 · PLANNING & DOCUMENTATION



FASE 2 · EXECUTION & DELIVERY



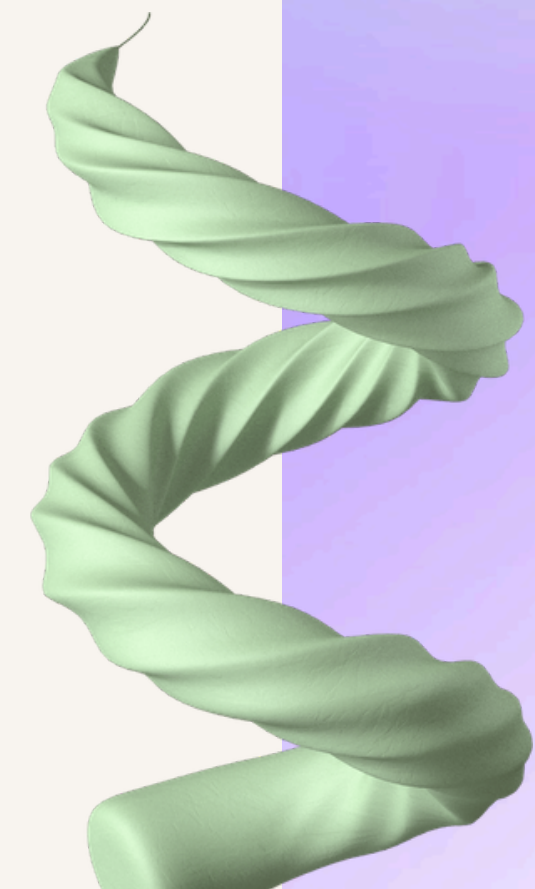
Software tidak lahir dari ide

Software lahir dari masalah yang sudah ada dan sudah merugikan seseorang. Kalau tidak ada yang dirugikan hari ini, kemungkinan besar tidak ada yang memakai aplikasi Anda besok.

✗ Aplikasi apa yang mau kita buat?

✓ Masalah siapa yang mau kita selesaikan?

MASALAH → KEBUTUHAN → FITUR → SOFTWARE



Cari masalahnya di Google Maps

Latihan paling cepat untuk menemukan masalah bisnis nyata: lihat bisnis yang benar-benar ada di sekitar kampus.

01 Buka Google Maps

Cari bisnis dalam radius 3 km dari kampus. Jangan cari ide, cari tempat.

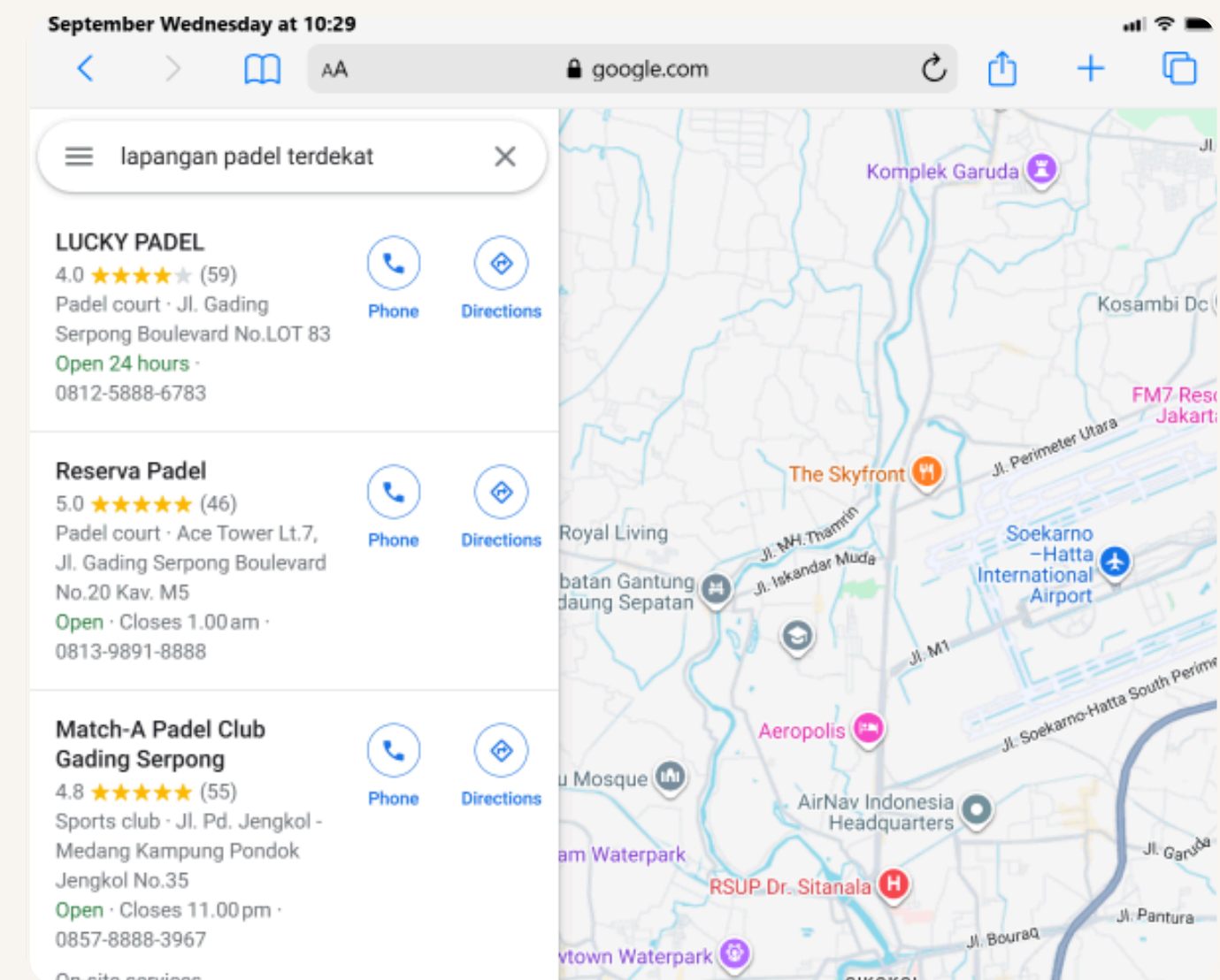
02 Pilih satu bisnis

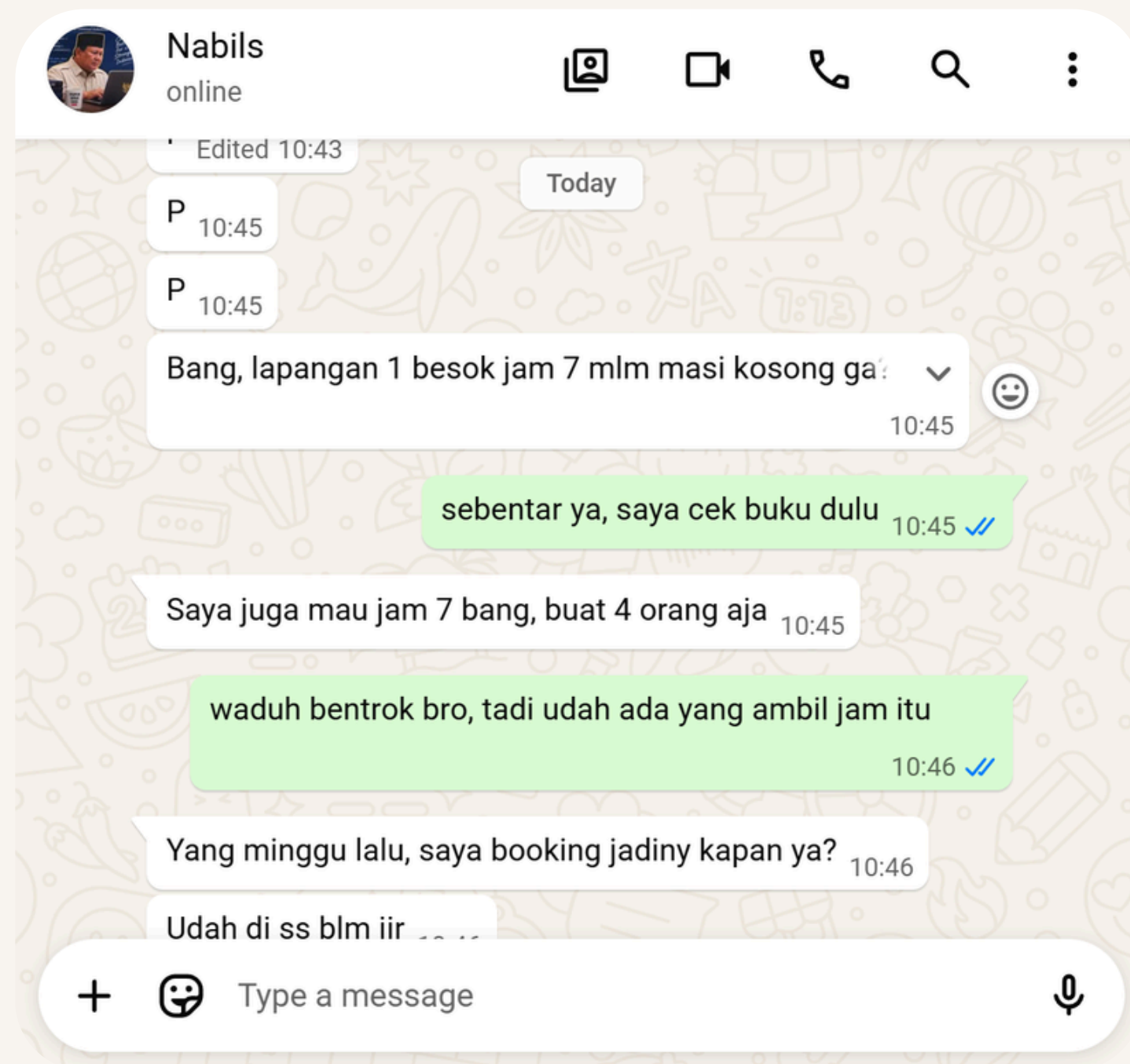
Utamakan yang operasionalnya masih manual: WhatsApp, buku tulis, atau spreadsheet.

03 Catat cara kerjanya

Bagaimana mereka menerima pesanan, mencatat, dan menagih hari ini?

Aturan main: pilih bisnis yang bisa Anda amati sendiri.





Masalahnya hidup di chat, bukan di sistem.

Studi kasus: Padel Court

Satu bisnis nyata yang akan kita bawa sampai production sore ini.

BISNIS

Padel Arena — penyewaan lapangan per jam

PELANGGAN

Komunitas dan pemain casual, ramai sore dan malam

BOOKING HARI INI

Chat WhatsApp ke admin, dicatat manual di buku

Cukup kecil untuk jadi MVP, cukup nyata untuk dipakai orang.

Merumuskan Problem Statement

Kumpulkan gejalanya dulu, baru rumuskan masalahnya dalam satu paragraf yang bisa diuji.

01 **Booking hanya lewat WhatsApp**

Tidak ada antrean, tidak ada bukti, semua bergantung admin.

02 **Jadwal sering bentrok**

Dua pelanggan bisa merasa sama-sama sudah memesan slot yang sama.

03 **Tidak ada kalender booking**

Pelanggan tidak bisa melihat slot kosong tanpa bertanya.

04 **Tidak ada dashboard admin**

Pemilik tidak tahu okupansi lapangan tanpa membuka buku catatan.

05 **Histori sulit dilacak**

Klaim pelanggan sulit diverifikasi karena tidak ada catatan digital.

BUSINESS PROBLEM STATEMENT

Padel Arena kehilangan pendapatan dan kepercayaan pelanggan karena seluruh proses booking berjalan lewat WhatsApp dan catatan manual. Tidak ada satu sumber kebenaran untuk jadwal, sehingga slot bisa terjual dua kali, riwayat pemesanan tidak bisa diverifikasi, dan pemilik tidak punya visibilitas atas okupansi lapangannya.

OUTPUT

PROBLEM.md

Documentation Driven Development

Sebelum satu baris kode ditulis, empat dokumen harus selesai. Dokumen inilah yang dibaca AI Agent — bukan pikiran Anda.

PRD.md

SCOPE.md

DESIGN.md

TASK.md

Kenapa dokumentasi menentukan hasil AI

AI Agent tidak membaca pikiran Anda. Ia membaca file.

TANPA DOKUMENTASI

✗ AI menebak kebutuhan

Setiap sesi menghasilkan asumsi baru yang saling bertabrakan.

✗ Tidak ada batas scope

Fitur terus bertambah karena tidak ada yang menyatakan cukup.

✗ Struktur berubah-ubah

Nama folder dan pola kode berbeda di tiap bagian aplikasi.

✗ Tidak bisa diaudit

Tidak ada acuan untuk menilai hasilnya benar atau salah.

DENGAN DOKUMENTASI

✓ AI mengeksekusi, bukan menebak

Kebutuhan sudah tertulis, tinggal diterjemahkan jadi kode.

✓ Scope punya rem

Yang tidak tertulis di scope tidak dikerjakan. Titik.

✓ Struktur konsisten

Desain dan penamaan mengikuti satu acuan yang sama.

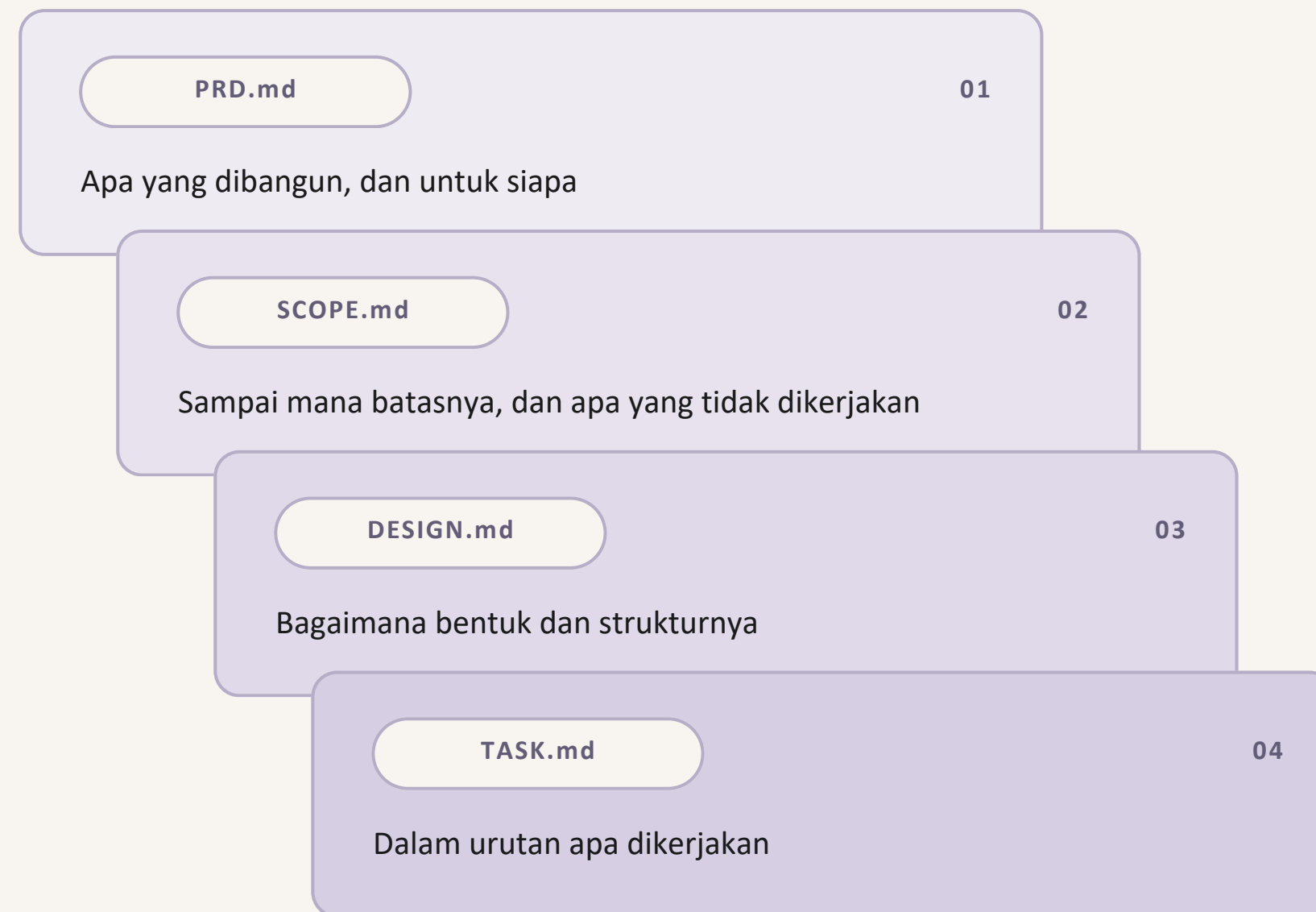
✓ Bisa diaudit siapa saja

Hasil kerja AI bisa dibandingkan langsung dengan dokumennya.

Dokumen ditulis lebih dulu. Kode menyusul sebagai konsekuensinya.

Documentation Stack

Empat dokumen yang menumpuk jadi satu blueprint. Inilah yang sebenarnya Anda serahkan ke AI Agent.



BLUEPRINT STACK

Empat lapis yang saling mengunci

Setiap lapis menjawab pertanyaan berbeda, dan setiap lapis membatasi lapis berikutnya. PRD menentukan isi Scope. Scope menentukan isi Design. Design menentukan isi Task. Kalau satu lapis dilewati, lapis di bawahnya kehilangan pijakan.

YANG DISERAHKAN KE AI AGENT

Empat file, satu folder. Itu seluruh konteksnya.

Sekarang kita buka IDE.

Empat template ini kita tulis langsung, bukan dibaca dari slide.

PRD.md

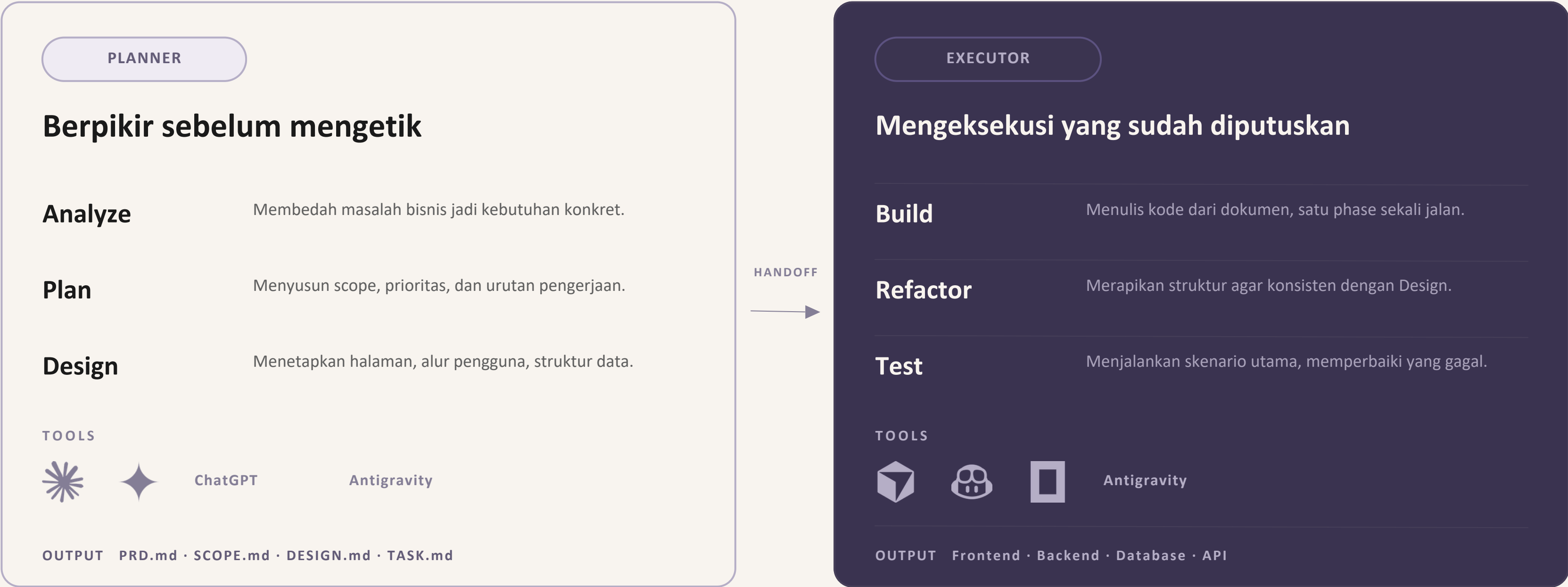
SCOPE.md

DESIGN.md

TASK.md

Planner vs Executor

Dua peran AI yang berbeda. Kesalahan paling umum adalah memakai satu AI untuk mengerjakan keduanya sekaligus.



Live Demo

Empat dokumen masuk. Satu aplikasi keluar. Yang menarik justru ada di bagian yang kita hentikan.

YANG AKAN TERJADI

- 01

Serahkan dokumen

PRD, Scope, Design, dan Task diberikan sebagai satu paket.
- 02

Jalankan per phase

Phase 1 sampai 4 dieksekusi berurutan, bukan sekaligus.
- 03

Hentikan dan baca

Setiap phase selesai, kita berhenti dan membaca hasilnya.
- 04

Jalankan aplikasinya

Booking dibuat dari browser, bukan dari chat WhatsApp.

Perhatikan berapa kali saya menghentikan AI, dan kenapa.

agent · padel-booking

N

TS

```
$ agent run --spec ./docs

reading PRD.md SCOPE.md DESIGN.md TASK.md


Phase 1  Project Setup

  [ok] Next.js + TypeScript

  [ok] Tailwind CSS

  [ok] Supabase client


Phase 2  Authentication

  [ok] Register · Login

  [...] Logout


Phase 3  Booking System

  [...] Create Booking
```

Human Review Checklist

AI menghasilkan kode yang jalan. Yang menentukan kode itu layak dipakai atau tidak, tetap Anda.



Product Review

Apakah yang jadi memang yang diminta?

- ☐ Sesuai PRD?
- ☐ Sesuai Scope?
- ☐ Ada fitur yang tidak diminta?
- ☐ User flow sesuai Design?



Technical Review

Apakah kodenya bisa dirawat bulan depan?

- ☐ Struktur folder rapi?
- ☐ Penamaan konsisten?
- ☐ Tidak over-engineering?
- ☐ Komponen dipakai ulang?



Security Review

Apakah aman dibuka ke internet?

- ☐ Tidak ada secret key di kode?
- ☐ Input tervalidasi?
- ☐ Akses admin terlindungi?
- ☐ Data sensitif tidak bocor?



Functionality Review

Apakah benar-benar berfungsi?

- ☐ Login berjalan?
- ☐ Booking berjalan?
- ☐ Dashboard berjalan?
- ☐ Error ditangani jelas?

Kalau tidak ada yang me-review, berarti pelanggan Anda yang jadi reviewer pertamanya.

Testing & Deployment

Uji jalur utamanya, lalu naikkan ke lingkungan yang bisa diakses orang lain.

HAPPY PATH

- ☐ **Login**
User terdaftar berhasil masuk dan diarahkan ke dashboard.
- ☐ **Booking**
Booking tersimpan dan slot yang sama tidak bisa dipesan lagi.
- ☐ **Approval**
Admin menyetujui booking dan statusnya berubah.
- ☐ **History**
Riwayat booking tampil lengkap dengan tanggal dan status.

Kalau salah satu gagal, jangan deploy.

DEPLOYMENT STACK



Key Takeaways

Lima hal yang saya harap tetap Anda ingat minggu depan, saat seminar ini sudah terlupakan.

- | | | |
|----|-----------------------------|--|
| 01 | Mulai dari masalah | Bukan dari ide, dan bukan dari tools. |
| 02 | Dokumen dulu | Kode adalah konsekuensi, bukan titik awal. |
| 03 | Scope adalah rem | Yang tidak tertulis, tidak dikerjakan. |
| 04 | AI mengeksekusi | Manusia yang tetap memutuskan. |
| 05 | Review itu pekerjaan | Bukan formalitas di akhir sprint. |



Q & A

AI Builder vs Agent, masa depan developer,
atau cara mulai belajar.



AI tidak menggantikan Software Engineer.

AI mempercepat Software Engineer
yang punya workflow yang benar.

Muhamad Rifki Firdaus

OR INSIGHT.